Vision - Guided Robotic Assistance for Spinal Interventions

Elena Blaco, Beatrice Andrea Mantegari, Maria Vittoria Sari
Biomedical Engineering
Politecnico di Milano, Milan, Italy
elena.blaco@mail.polimi.it, beatriceandrea.mantegari@mail.polimi.it, mariavittoria.sari@mail.polimi.it

***Abstract*—Lumbar spinal stenosis is a degenerative condition that most commonly affects the mobile L4–L5 segment. Among the available treatment techniques, lumbar decompression surgery, and particularly laminotomy supported by robotic systems, offers several advantages in precision and repeatability. In this context, this work focuses on the L4–L5 laminotomy region, combining real-time AI-based anatomical object detection with robotic guidance. After image acquisition, an object detection model based on YOLOv8 was trained to localize the region of interest in real time and compute the corresponding centroids. A hand–eye calibration procedure was then performed and validated to enable the transformation of points from the camera reference system to that of the robot base. This information was integrated to simulate the robotic system and support the evaluation of the adopted control strategy. The results show high accuracy for the model trained for object recognition in identifying the area of interest (L4–L5 laminae), while validation of the hand–eye calibration procedure shows a low standard deviation in error metrics, confirming the repeatability and stability of the procedure. The movements of the robotic system in the simulated environment are consistent with the planned trajectories, validating the entire workflow. The quantitative and qualitative results form the basis for the subsequent implementation and testing phases on the actual robot.**

***Index Terms*—Lumbar spinal stenosis, laminotomy, robotic guidance, hand–eye calibration, object detection, YOLOv8**

I. Introduction

Lumbar spinal stenosis (LSS) is a degenerative condition in which the spinal canal gradually becomes narrower, reducing the space available for the nerve roots and leading to mobility limitations, leg pain, and reduced walking tolerance [1]. It is among the most common causes of lower-back–related disability in adults over 50, and epidemiological studies show a marked increase in its prevalence with aging [2]. Degenerative changes most often affect the L4-L5 segment, which is the most mobile level of the lumbar spine and therefore the one most frequently involved in stenosis and related neurological symptoms. As a result, L4-L5 is also the level most commonly targeted for decompression surgery [3]. Treatment usually begins with conservative approaches but when symptoms persist and significantly impact daily activities, lumbar decompression surgery becomes indicated. Among available techniques, lumbar laminotomy is a widely used procedure in which only a small part of the lamina is removed to create additional space in the canal. Compared with full laminectomy, laminotomy preserves more stabilizing structures and is associated with reduced tissue disruption, fewer complications, and faster recovery, while still providing significant improvements in pain and function [4]. Nevertheless, laminotomy demands high precision: small deviations in tool placement can lead to incomplete decompression or unintended contact with nearby tissues. Robotic systems can assist the surgeon by stabilizing instrument motion, reducing the effects of hand tremor and fatigue, and potentially enabling smaller incisions and more consistent outcomes. Studies have demonstrated that, once adopted, these systems can achieve a high degree of accuracy and precision, particularly in tasks that require meticulous spatial control [5]. In line with these developments, this work focuses on the L4-L5 laminotomy region, combining real-time AI-based anatomical detection with robotic guidance. This paper presents the system architecture, including the vision pipeline, calibration workflow, coordinate transformation strategy and simulator control, and provides initial validation demonstrating the feasibility of integrating AI-based localization with robotic assistance for lumbar L4-L5 laminotomy.

II. State of Art

Recent advancements in robot-assisted spinal surgery have focused on improving precision and reducing risks. In this context, laminotomy provides a minimally invasive approach that allows for targeted decompression while preserving more surrounding tissue. Laminotomy is a procedure that involves excising part of the lamina and ligamentum flavum to visualise the stenotic area and safely undercut the medial facet, preserving more of the surrounding tissues and muscle compared to laminectomy, which removes the entire lamina [4]. In spinal surgery, robot-assisted technologies have primarily been used for pedicle screw placement, significantly improving the precision of screw insertion and reducing radiation exposure. Studies show that robotic assistance leads to a significant reduction in misplaced screws when compared to free-hand or fluoroscopy-assisted techniques [6]. From a technological standpoint, the literature on robot-assisted laminar bone removal has mainly emphasized image-based planning and spatial registration to guide robotic execution in decompression tasks. Systematic analyses report that most robotic systems rely on preoperative three-dimensional medical imaging to reconstruct vertebral anatomy, define the laminar region of interest, and generate constrained paths for robot-assisted cutting, while also stressing that robust spatial registration

and calibration between imaging data and the robot reference frame are critical prerequisites for accurate execution [7]. Recent studies have highlighted the use of real-time image-guided robotic assistance in spine surgery, particularly in the placement of pedicle screws. One key application involves the integration of intraoperative imaging, such as 3D cone-beam CT, with robotic systems like the ExcelsiusGPS™ robot. This system allows surgeons to visualize the surgical site in real time, ensuring that the robot's guidance remains accurate and continuously aligned with the patient's anatomy. The real-time feedback significantly reduces the risk of misplacement, enhancing screw accuracy, while also providing continuous spatial awareness during the procedure. Additionally, this approach helps reduce surgeon fatigue and decreases the overall radiation exposure compared to traditional fluoroscopic methods [8]. On the other hand, our project utilizes RGB-D sensing as an alternative to traditional medical-image-based approaches, enabling immediate detection of the region of interest. Real-time RGB-D acquisition has demonstrated the ability to support intraoperative reconstruction and registration of spinal surfaces with millimetric precision, facilitating vision-driven localization without relying solely on preoperative imaging [9].

III. Material and Methods

A. Workflow

As shown in Fig. 1, this section describes the overall workflow adopted for the proposed vision-guided robotic framework. The pipeline begins with the acquisition of images of the spinal phantom, *Posterior Lumbar Box – Upsurgeon*, followed by dataset preparation and annotation for training the object detection model. The trained detector is then employed to localize the L4–L5 region of interest in real time. The detected region is subsequently used to compute the 3D centroid of the target anatomy. A hand–eye calibration procedure is then performed and validated to establish the spatial relationship between the camera and the robot. Finally, the estimated 3D target position, expressed in the robot reference frame through the validated hand–eye calibration, is used to simulate the robot motion toward the identified anatomical target. In this way, the simulated robot movement is directly guided by the visual information acquired from the camera, allowing the evaluation of the control strategy within a vision-guided configuration.

B. Image Acquisition

Images data were acquired using an Intel RealSense D435 RGB-D camera [10], configured and controlled through a custom Python script developed for this study. The D435 is equipped with an RGB sensor capable of capturing color frames up to 1920×1080 at 30 FPS, and a stereo depth module with global-shutter IR sensors offering a depth field of view of approximately 87° × 58° and a working range extending from about 0.3 m up to 10 m. For this study, the camera was set to acquire RGB frames at a resolution of 640×480 and 30 FPS. This resolution was selected as a trade-off between image detail and computational efficiency, allowing faster processing and real-time performance of the object detection model while maintaining sufficient spatial information for reliable localization of the target anatomy. Auto-exposure was enabled when available. The script handled camera initialization, exposure management, real-time visualization, and image storage. A total of 279 RGB images were manually captured by the operator and saved in JPEG format. The acquisition was performed in different areas of the classroom to introduce natural variations in ambient illumination, including brighter and dimmer conditions as well as localized shadows. To further increase dataset variability, the phantom was placed on different backgrounds and recorded from multiple viewpoints and distances, ensuring a broad range of visual conditions. Additionally, unrelated objects were intentionally positioned around the target structure to increase scene complexity and promote model robustness and generalization. Each acquired frame was processed with a sharpening filter to enhance edge definition prior to storage. The resulting dataset, characterized by variability in illumination, background, object placement, and scene clutter, constituted the raw data used for training and validating the object detection mode.

C. Data Preparation

Following acquisition, all 279 RGB images were uploaded to Roboflow, which was used as the primary platform for dataset management, annotation, preprocessing, and export. A single object class, Laminae (class 0), was defined to label the L4–L5 region of interest. Manual bounding box annotation was performed on the training and validation subsets only, resulting in a total of 237 annotated images. The remaining 42 images were reserved as an independent test set and were not annotated, as they were used exclusively for final model evaluation. The dataset was split into 72% for training (201 images), 13% for validation (36 images), and 15% for testing (42 images). Roboflow was further used to generate a YOLOv8-compatible dataset structure for subsequent training. The acquisition procedure intentionally introduced variability in target position, scale, orientation, illumination conditions, and background composition. As a result, the annotated bounding boxes exhibit a broad spatial distribution across the image plane and moderate variability in size and aspect ratio, reflecting differences in viewpoint and distance. Additionally, the presence of distractor objects and heterogeneous scene conditions was maintained to encourage model robustness and generalization. Overall, the dataset preparation process resulted in a structured and diverse dataset suitable for training and evaluating an object detection model for L4–L5 laminae localization.

D. Object Detection of L4-L5 ROI

Object detection was performed using YOLOv8 [11], trained using the dataset exported from Roboflow. The training process was implemented through a dedicated Python script, employing the pretrained yolov8s.pt model. Two training configurations were tested: 25 epochs and 50 epochs,

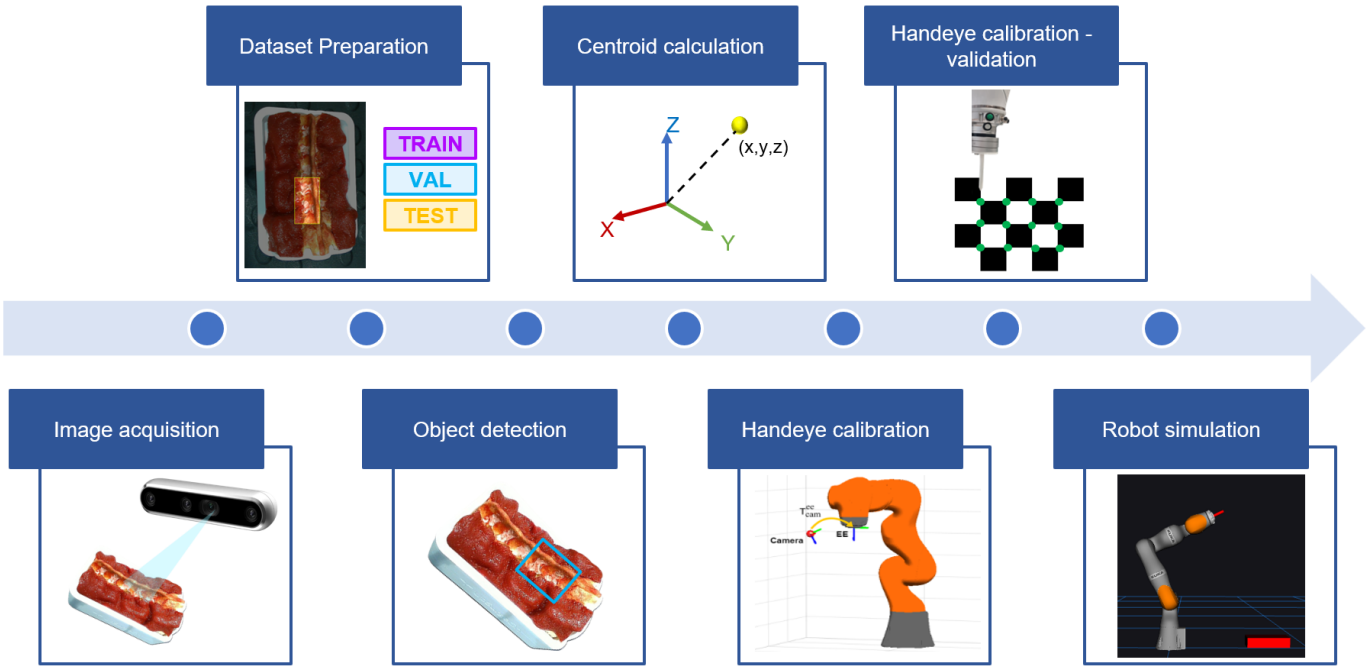

Fig. 1. Block diagram of the proposed workflow

which achieved superior performance. Training was conducted with a batch size of 16. YOLOv8 computes several key performance metrics during validation, including: precision, recall, mAP@0.5 (mean Average Precision at IoU = 0.5) and mAP@0.5:0.95 (average across IoU thresholds from 0.5 to 0.95). The Object detection aims to produce boundary boxes enclosing the entire L4-L5 intervertebral complex. For real-time inference, a custom Python application was developed to interface the YOLOv8 detector with the Intel RealSense D435 RGB-D camera, enabling synchronized color and depth acquisition at 640×480 resolution and 30 FPS. The detection pipeline was extended to compute the 2D centroid of the predicted bounding box and to estimate the corresponding 3D position. The 2D centroid of the predicted bounding box was computed from the detected pixel coordinates as:

$$u_c = \frac{x_{\min} + x_{\max}}{2}, \qquad v_c = \frac{y_{\min} + y_{\max}}{2} \tag{1}$$

where $(x_{\min}, y_{\min})$ and $(x_{\max}, y_{\max})$ denote the bounding box corners in image coordinates. The corresponding depth value was extracted from the aligned depth frame:

$$Z = D(u_c, v_c) \tag{2}$$

where $D(\cdot)$ represents the depth image and Z is expressed in meters. The 3D position of the detected object in the camera reference frame was obtained by deprojecting the pixel coordinates using the intrinsic camera model, with the depth value directly measured by the RGB-D sensor. Given the intrinsic parameters f_x, f_y (focal lengths) and c_x, c_y (principal point coordinates), the Cartesian coordinates were computed as:

$$X = \frac{(u_c - c_x)Z}{f_x}, \qquad Y = \frac{(v_c - c_y)Z}{f_y}, \qquad Z = Z \tag{3}$$

Thus, the reconstructed 3D point in the camera reference frame is:

$$\mathbf{p}_C = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = Z \begin{bmatrix} \frac{u_c - c_x}{f_x} \\ \frac{v_c - c_y}{f_y} \\ 1 \end{bmatrix} \tag{4}$$

This operation corresponds to the inversion of the intrinsic projection model:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} X/Z \\ Y/Z \\ 1 \end{bmatrix} \tag{5}$$

where the intrinsic calibration matrix is defined as:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{6}$$

To improve robustness against sensor noise, the estimated 3D position was further stabilized through a lightweight Kalman filter implemented in Python. In addition to real-time detection, the custom Python code included an acquisition module that allowed the operator to manually capture RGB frames together with the corresponding filtered 3D centroid coordinates.

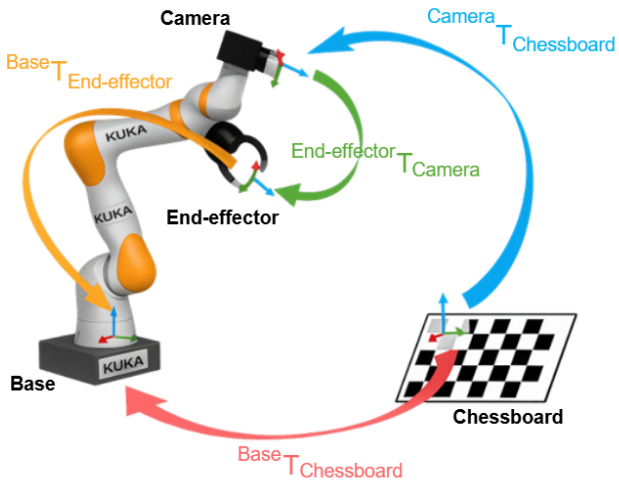

Fig. 2. Transformation chain involved in the hand–eye calibration.

E. Hand–Eye Calibration

The hand–eye calibration procedure was carried out using a dataset composed of 80 images of a calibration chessboard and the corresponding robot poses. Each image is associated with an end–effector pose expressed in terms of Cartesian position and quaternion orientation with respect to the robot base frame. Figure 2 illustrates the reference frames involved in the hand–eye calibration procedure and the transformations estimated from the acquired dataset.

Prior to performing the hand–eye calibration, the intrinsic calibration of the camera was carried out. This calibration was performed using the chessboard images, exploiting the known geometry of the calibration pattern. The chessboard consists of an arrangement of 11×8 squares, corresponding to 10×7 inner corners, with a square side length of 24 mm. The calibration procedure was implemented in Python using the OpenCV library. For each image, the chessboard detection was first performed. The images were converted to grayscale and preprocessed using Contrast Limited Adaptive Histogram Equalization (CLAHE) and Gaussian filtering to enhance contrast and reduce noise. The inner corners of the chessboard were then detected using the `findChessboardCornersSB` algorithm. The detected 2D image points were subsequently associated with the corresponding 3D object points of the chessboard, defined according to the known geometry of the calibration pattern. These 2D–3D correspondences were used to estimate the intrinsic camera parameters, namely the camera matrix and the radial and tangential distortion coefficients. The intrinsic parameters were estimated using the forward projection model relating 3D object points to their corresponding 2D image projections:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \tag{7}$$

The intrinsic calibration was performed using the `calibrateCamera` function, which solves a nonlinear optimization problem by minimizing the reprojection error between the observed image points and the points projected by the camera model. To improve the robustness of the estimation, the pose of the chessboard with respect to the camera was additionally estimated for each valid image using the OpenCV `solvePnP` function, thus obtaining the extrinsic parameters (rotation and translation) of the camera with respect to the calibration target. The reprojection error was computed for each image, and images with a per-image reprojection RMSE greater than 0.50 pixels were discarded. After this filtering step, 56 images were retained and used to compute the final intrinsic parameters. Subsequently, the camera–to–chessboard poses were estimated by computing, for each valid image, the rigid transformation between the chessboard frame and the camera frame. These transformations represent the target–to–camera poses required for the hand–eye calibration. The hand–eye calibration was then performed using a dedicated Python implementation. The objective was to estimate the rigid transformation between the camera frame and the robot end–effector frame. For each valid acquisition, the robot poses, originally expressed in quaternion form, were converted into rotation matrices, allowing the construction of the gripper–to–base transformations. In parallel, the target–to–camera transformations were obtained from the intrinsic and extrinsic camera calibration results. The hand–eye transformation was estimated using the OpenCV `calibrateHandEye` function, adopting the Tsai–Lenz method, which solves the classical hand–eye calibration problem:

$$A_i X = X B_i, \tag{8}$$

where A_i represents the motion of the end–effector in the robot base frame, B_i represents the motion of the chessboard in the camera frame, and X is the unknown transformation between the camera and the end–effector. The procedure outputs the camera–to–end–effector transformation matrix, which is subsequently used in the following stages of the project.

F. Handeye–Calibration Validation

The validation of the hand–eye calibration was performed using a KUKA LBR iiwa 7 robotic system equipped with the Intel RealSense RGB camera mounted on the end–effector and a software infrastructure based on ROS 2. The entire workflow was executed within a virtual machine running Ubuntu 22.04. The validation procedure was carried out using provided ROS 2 scripts, which were adapted by inserting the hand–eye transformation matrix estimated in the previous phase. Data acquisition was performed in a synchronized manner between RGB images and the robot end–effector pose. For each acquisition, the code collects 20 consecutive samples of the robot pose, acquired with a time interval of 0.03 s, corresponding to a sampling frequency of approximately 33 Hz. The poses are averaged to reduce measurement noise and associated

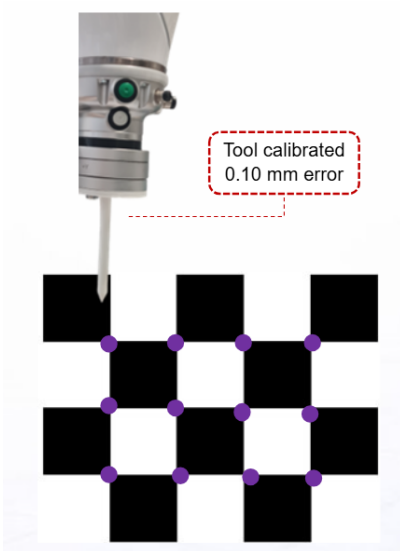

Fig. 3. Geometric validation procedure of the hand–eye calibration.

with the acquired image of the calibration chessboard. The data are saved as image files and in a CSV file containing the filtered poses. To validate the calibration (Fig. 3), 12 ground truth points were provided, and the corresponding points were reconstructed using the chain of transformations to be validated.

Specifically, the ground truth points were acquired from a chessboard different from the one used to reconstruct $T_{\text{cam}\to\text{ee}}$. The reference points were identified using a rigid stick attached to the base of the robot, which was calibrated with 4TCP (4–Point Tool Center Point) calibration. This procedure provides the transformation of the tool with respect to the end effector and thus determines where the tool points with respect to the base of the robot. Instead, the reconstructed points were estimated from images of the same chessboard used to identify the ground truths. Geometric validation to confirm the correctness of the transformation obtained through hand–eye calibration is performed for each point using an automatic point-to-point comparison between corresponding points in the two sets. The Euclidean distance between each pair of corresponding points is then calculated, and statistical error metrics are obtained. Both reconstructed and reference points are visualized in RViz using 3D markers, allowing qualitative assessment of spatial alignment within the robot base reference system.

G. Robot Simulation

The simulation aims to create a virtual environment that closely replicates real behavior. Specifically, it was used to test the execution of desired trajectories, verify the correctness of joint configurations, and identify any critical issues before operating the physical robot. The robot considered in the simulation is the KUKA LBR iiwa 7, introduced in the previous subsections. It is a seven-degree-of-freedom manipulator designed for applications requiring high precision and safe interaction with the environment. The structure of its seven revolute joints provides kinematic redundancy, making the robot suitable for executing complex trajectories and precise movement control. In this context, the robot was implemented in the virtual environment using the model provided by the lbr-stack software stack [12] on the GitHub cloud platform. As anticipated, the simulated environment was set up on a Linux system within a virtual machine to ensure compatibility with the required software tools. These tools include ROS 2 Humble, the main framework for managing nodes, communication, and robot control; lbr-stack (lbr-fri-ros2-stack), as previously mentioned, for integrating the KUKA LBR iiwa 7 robot model and control interfaces; RViz 2, for monitoring and visualizing the robot's status; MoveIt 2, for movement planning and joint management; and Python, for developing custom nodes to plan and control the desired movements. The simulation was structured using a modular approach based on ROS 2 nodes. Specifically, trajectories were defined and executed from the desired poses, which were appropriately processed and then sent to the robot. In the first phase, 13 different poses were defined and associated with the spatial positions of the centroids relevant to the task, since these represent the kinematic objectives of the movement. This information was collected using the Python code already described in the subsection Object detection in the L4-L5 ROI and was stored in the `captures.csv` file, whose format was chosen for its compatibility with ROS 2 nodes. These data are used as input in a ROS 2 node developed in Python, allowing them to be read and processed for interpretation by the control system. Starting from the Cartesian poses, the node calculates the inverse kinematics using the model provided by the simulation environment. Based on this processing, commands are sent to execute the desired movement. The context of spinal surgery led to the choice of translational movement along the xy plane, followed by a perpendicular descent above the target point. The 3D coordinates of the target point are transformed from the camera reference frame to the robot base frame. This transformation is performed by combining the hand–eye calibration matrix $T_{\text{cam}\to\text{ee}}$ with the end–effector pose $T_{\text{ee}\to\text{base}}$ reconstructed from position and quaternion data stored in the CSV file. The chain of transformations described above is shown in the equation below and can be seen in its entirety in Figure 2.

$$^{base}T_{cam} = {}^{base}T_{ee}\ {}^{ee}T_{cam} \tag{9}$$

The execution of the planned trajectories is monitored using Rviz 2, which allows the robot, joint configurations, and overall movement to be visualized. This final phase enables the identification of any errors, inconsistencies, or inadmissible configurations, contributing to the validation of the simulation method.

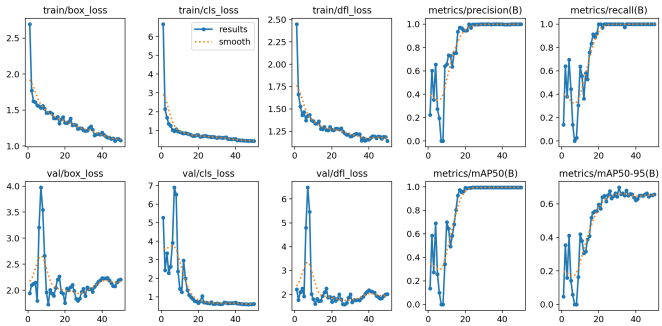

Fig. 4. Statistics results of the YOLOv8 object detection model.

IV. Results

A. Object detection of L4-L5

The YOLOv8-based object detection model was trained using the previously described annotated data set, with configurations of 25 and 50 epochs. The 50-epoch configuration yielded superior performance and was therefore selected for the final model (Fig. 4). During training, the loss curves showed stable convergence across all components. Box loss, classification loss, and distribution focal loss progressively decreased over the epochs, while validation losses followed the same trend, indicating consistent learning without signs of overfitting. The mAP metrics also exhibited a rapid increase during the initial epochs before stabilizing at values close to their maximum, confirming that the model quickly learned to distinguish the L4–L5 laminae region.

The final validation metrics demonstrate that the model achieved near-perfect performance. The mAP@0.5 reached 0.995, while the mAP@0.5:0.95 plateaued at approximately 0.67, which is consistent with typical behavior for small, anatomically constrained targets. Precision and recall both converged to 1.00, as also reflected in the precision–recall curve, which exhibits an almost ideal rectangular shape. The confusion matrix confirms that the detector correctly identified all instances of the *Laminae* class in the test set, with no false positives or false negatives. Overall, the detection model achieved high accuracy in localizing the L4–L5 region of interest, demonstrating that the combination of a well-constructed dataset and the YOLOv8 architecture is effective for this clinical imaging application. These results also validate the suitability of the model for real-time inference within the custom detection pipeline. In the real-time implementation, the system successfully provided both the 2D bounding box and the corresponding 3D centroid coordinates, which were further stabilized using a Kalman filter to compensate for sensor noise. The detector maintained real-time performance at 30 FPS and consistently produced accurate localization information suitable for robot-control simulation tasks (Fig. 5).

However, testing also revealed an important limitation of the depth sensing hardware. Intel RealSense D435 operates reliably only within its effective depth range, whose minimum working distance is approximately 18 cm at the selected resolution. When the camera was positioned closer than this threshold to the laminae, the depth sensor was no longer able to accurately reconstruct the z-coordinate. Under these conditions, the stereo-based depth estimation becomes unstable and may return invalid or zero-valued measurements, preventing the system from computing a valid 3D centroid. This limitation in close-range depth reconstruction had to be considered during the robot control phase, ensuring that the camera remained within its effective operating range to guarantee stable 3D centroid estimation.

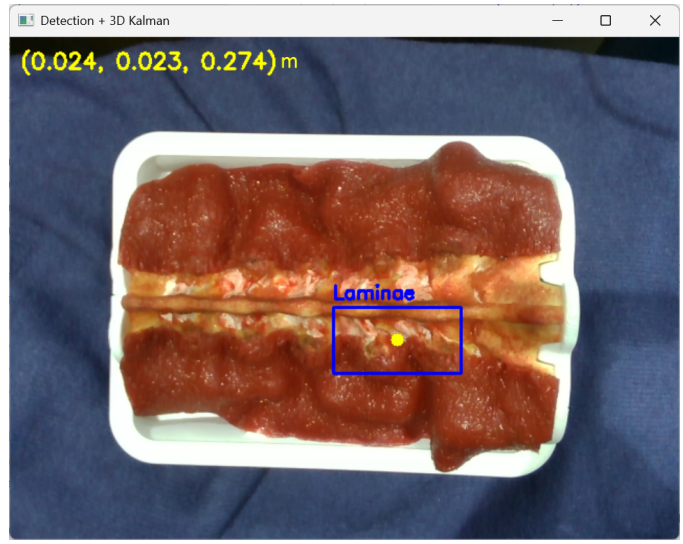

Fig. 5. Object Detection and Centroid Calculation.

B. Hand–eye Calibration

After applying the filtering procedure described in the Method section, 56 valid images were retained and used to compute the refined intrinsic parameters.The resulting intrinsic calibration matrix is:

$$\mathbf{K}_{\text{final}} = \begin{bmatrix} 654.48 & 0 & 635.90 \\ 0 & 653.61 & 391.95 \\ 0 & 0 & 1 \end{bmatrix}$$

and distortion coefficients:

$$\mathbf{d}_{\text{final}} = \begin{bmatrix} -0.04395 & 0.06806 & 0.00675 & -0.00206 & -0.02625 \end{bmatrix}$$

After filtering, the global reprojection error decreased to 0.349 px, indicating an overall improvement in calibration accuracy.

It can be noticed that the estimated intrinsic parameters differ in scale from the nominal intrinsics provided by the RealSense device, which are specified for a resolution of 640×480. Since the calibration was performed using images with resolution 1280×720, the focal lengths and principal point coordinates are consistently scaled according to the actual image resolution used during acquisition.

For each of the 56 valid images, the rigid transformation between the chessboard frame and the camera frame was

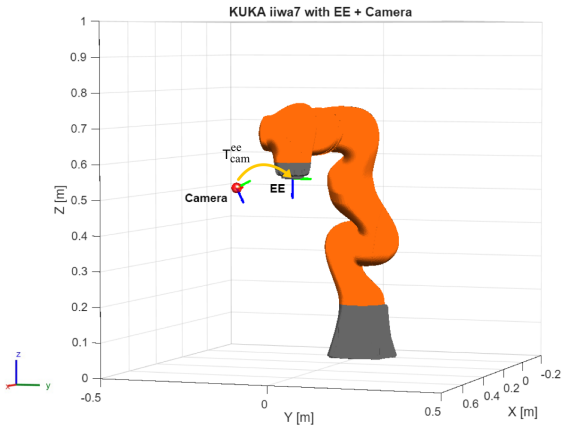

Fig. 6. Visualization of the KUKA iiwa7 robot with the estimated end–effector and camera frames in MATLAB.

estimated by solving a Perspective–n–Point (PnP) problem. These transformations represent the target–to–camera poses required for the hand–eye calibration and were stored in the file `extrinsics_per_image_filtered.csv`. The generated images show a consistent alignment between the estimated chessboard reference frame and the observed pattern, confirming the geometric consistency of the estimated extrinsic parameters. Using the robot end–effector poses and the camera–to–chessboard transformations, the hand–eye calibration procedure was performed. The method estimated the rigid transformation between the camera frame and the robot end–effector frame, expressed in meters. The resulting camera–to–end–effector transformation matrix is:

$$\mathbf{T}_{\text{cam}\to\text{ee}} = \begin{bmatrix} 0.99997 & -0.00314 & -0.00725 & 0.00858 \\ 0.00607 & 0.89345 & 0.44913 & -0.44913 \\ 0.01713 & -0.44960 & 0.89344 & 0.02955 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The inverse transformation, representing the pose of the end–effector with respect to the camera, was also computed and saved for further use.

A qualitative validation of the hand–eye calibration results was carried out by visualizing the robot and the estimated camera pose in MATLAB. The KUKA iiwa7 robot model was displayed together with the end–effector frame and the camera frame positioned according to the estimated transformation. Figure 6 shows the resulting visualization, highlighting a geometrically consistent placement of the camera with respect to the end–effector, in agreement with the physical configuration of the experimental setup.

C. Hand–eye Calibration Validation

As mentioned above, calibration validation was performed using 12 corners of a chessboard measured in two different ways. To establish the accuracy of the calibration, a comparison was made between the two sets of points obtained based on Euclidean distance, the equation for which is shown below.

TABLE I
QUANTITATIVE RESULTS OF THE HAND–EYE CALIBRATION VALIDATION.

Metric	Value
Number of points	12
Mean error $\lvert\mathbf{e}\rvert$	27.71 ± 0.39 mm
Median error $\lvert\mathbf{e}\rvert$	27.71 mm
RMSE $\lvert\mathbf{e}\rvert$	27.72 mm
Maximum error	28.38 mm
Mean error along x	8.88 ± 0.90 mm
Mean error along y	0.46 ± 1.21 mm
Mean error along z	-26.20 ± 0.44 mm

$$d_i = \|\mathbf{p}_i^{\text{ref}} - \mathbf{p}_i^{\text{rec}}\| = \sqrt{(x_i^{\text{ref}} - x_i^{\text{rec}})^2 + (y_i^{\text{ref}} - y_i^{\text{rec}})^2 + (z_i^{\text{ref}} - z_i^{\text{rec}})^2} \tag{10}$$

Table I summarizes the main error metrics obtained from the validation procedure.

The quantitative results show a mean Euclidean error of approximately 28 mm across the reconstructed chessboard points, with a very low standard deviation. This indicates that the error is highly consistent among all points, suggesting a systematic component rather than random noise. The axis–wise error analysis shows a significantly larger error component along the robot z–axis, on the order of approximately 2 cm, while the errors along the x and y axes remain comparatively smaller. This behavior can be explained by the limited excitation of the z direction during the hand–eye calibration procedure. In particular, the set of robot poses used for calibration did not sufficiently span the z axis, leading to a reduced observability of the depth component and, consequently, to a larger systematic error along this direction. During the data acquisition phase, the robot end–effector pose was not obtained from a single measurement but computed as the average of multiple samples acquired from the ROS topic providing the end–effector pose with respect to the robot base. This averaging process, performed at a known sampling frequency, was introduced to reduce measurement noise and improve repeatability. While, this approach effectively mitigates random noise, it does not compensate for systematic errors arising from limited pose diversity, which predominantly affect the z axis.

Overall, the results indicate that the calibration is repeatable and stable, as confirmed by the low standard deviation of the error metrics, but that the accuracy along the depth direction is constrained by the pose distribution used during the hand–eye calibration.

The qualitative validation of the calibration was carried out through visualization in RViz 2. Figure 7 shows the robot model together with the reconstructed chessboard points and the reference points, all expressed in the robot base frame.

D. Robot Simulation

In the overall context of the project, simulation constitutes a preliminary and essential phase for validating movement strategies. The positive results obtained in the virtual environment are qualitative and have enabled adopted workflow

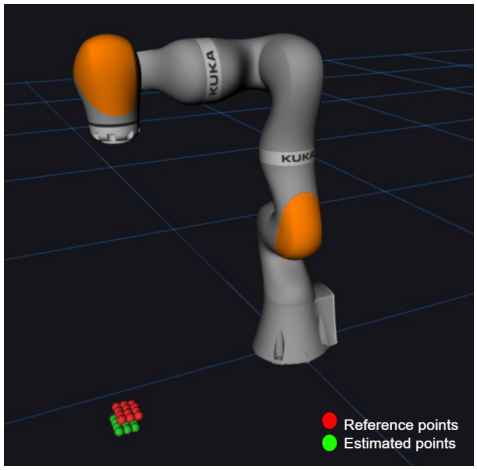

Fig. 7. RViz 2 visualization of the hand–eye calibration validation. The robot model is shown together with the reconstructed chessboard points and the reference points in the robot base frame.

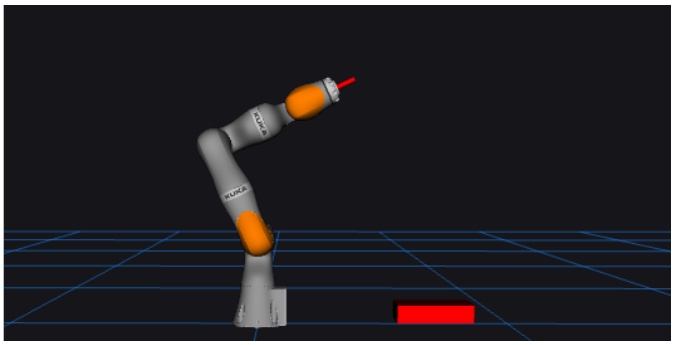

Fig. 8. RViz 2 visualization of KUKA iiwa 7, surgical tool attached to the end–effector and schematic model of the phantom.

validation. At the same time, they provide the basis for the subsequent implementation and testing phases on the actual robot. Figure 8 shows the simulated environment, in which there are the KUKA iiwa 7 robot model, whose end-effector has been fitted with a cylindrical instrument representing a generic surgical instrument and also a red box representing the phantom used in this work.

V. Conclusions

This work presented a vision-guided robotic assistance pipeline for lumbar L4–L5 laminotomy, integrating real-time anatomical localization with a calibrated camera–robot spatial relationship and a simulation-based validation stage. The proposed framework combines RGB-D acquisition, YOLOv8-based detection of the laminae region, 3D centroid estimation, and a hand–eye calibration procedure enabling the conversion of visually detected targets into the robot base frame. Experimental results support the feasibility of the approach. The detector achieved near-perfect performance (mAP@0.5 = 0.995) and maintained real-time operation at 30 FPS, providing stable centroid estimates for downstream robotic tasks. Camera calibration benefited from filtering high-error images, reducing the global reprojection error to 0.349 px; however, depth accuracy remains sensitive to the RGB-D operating range, especially when the camera is positioned too close to the target. Hand–eye validation revealed a clear systematic error along the z direction (mean error along $z = -26.20 \pm 0.44\,\mathrm{mm}$), which is consistent with the limited excitation of the z-axis during calibration and with the intrinsic limitations of depth sensing. In simulation, the complete transformation chain enabled coherent robot motions, providing a solid baseline for transferring the method to a physical setup.

Future work will prioritize improving absolute accuracy through a more diverse calibration pose set (with stronger depth excitation), more robust handling of the depth direction, extend this application beyond simulation, and deploy it in a real surgical setting. Respiratory motion could also be compensated for by having the camera–robot system track the target in real time and close the loop at its continuously updated position. This would remove the current static-centroid assumption and keep the estimated centroid consistently registered to the phantom.

References

[1] J. N. Katz, Z. E. Zimmerman, H. Mass, and M. C. Makhni, "Diagnosis and management of lumbar spinal stenosis: a review," *Jama*, vol. 327, no. 17, pp. 1688–1699, 2022.

[2] L. Kalichman, R. Cole, D. H. Kim, L. Li, P. Suri, A. Guermazi, and D. J. Hunter, "Spinal stenosis prevalence and association with symptoms: the framingham study," *The spine journal*, vol. 9, no. 7, pp. 545–550, 2009.

[3] R. A. Deyo, S. K. Mirza, and B. I. Martin, "Back pain prevalence and visit rates: estimates from us national surveys, 2002," *Spine*, vol. 31, no. 23, pp. 2724–2727, 2006.

[4] M. G. Williams, A. M. Wafai, and M. D. Podmore, "Functional outcomes of laminectomy and laminotomy for the surgical management lumbar spine stenosis," *Journal of Spine Surgery*, vol. 3, no. 4, p. 580, 2017.

[5] J. L. Goldberg, R. Härtl, and E. Elowitz, "Minimally invasive spine surgery: an overview," *World Neurosurgery*, vol. 163, pp. 214–227, 2022.

[6] S. N. D. D. K. C. G. G. D. S. Matthew S. Galetta, Joseph D. Leider, "Robotics in spinal surgery," *Annals of Translational Medicine*, 2019.

[7] Z. Li, G. Yu, S. Jiang, L. Hu, and W. Li, "Robot-assisted laminectomy in spinal surgery: a systematic review," *Annals of translational medicine*, vol. 9, no. 8, p. 715, 2021.

[8] A. K. Ahmed, C. C. Zygourakis, S. Kalb, A. M. Zhu, C. A. Molina, B. Jiang, A. M. Blitz, A. Bydon, N. R. Crawford, and N. Theodore, "First spine surgery utilizing real-time image-guided robotic assistance," *Computer Assisted Surgery*, vol. 24, no. 1, pp. 13–17, 2019.

[9] F. Liebmann, D. Stütz, D. Suter, S. Jecklin, J. G. Snedeker, M. Farshad, P. Fürnstahl, and H. Esfandiari, "Spinedepth: a multi-modal data collection approach for automatic labelling and intraoperative spinal shape reconstruction based on rgb-d data," *Journal of Imaging*, vol. 7, no. 9, p. 164, 2021.

[10] Intel Corporation, "Intel® RealSense™ Depth Camera D435," https://www.realsenseai.com/products/stereo-depth-camera-d435/, 2023.

[11] Ultralytics, "YOLOv8," https://yolov8.com/, 2023.

[12] M. Huber, C. E. Mower, S. Ourselin, T. Vercauteren, and C. Bergeles, "LBR-stack: ROS 2 and python integration of KUKA FRI forMed and IIWA robots," vol. 9, no. 103, p. 6138. [Online]. Available: https://joss.theoj.org/papers/10.21105/joss.06138